ISA Transactions 145 (2024) 479–492

Contents lists available at ScienceDirect

ISA Transactions

journal homepage: www.elsevier.com/locate/isatrans

Practice article

Combined control algorithm based on synchronous reinforcement learning for a self-balancing bicycle robot

Lei Guo [a,*], Hongyu Lin [a], Jiale Jiang [a], Yuan Song [a], Dongming Gan [b]

[a] School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China
[b] School of Engineering Technology, Purdue University, West Lafayette, IN, USA

ARTICLE INFO

Keywords:
Bicycle robot
Dynamics model
Nonaffine nonlinear system
Synchronous reinforcement learning
Combined control algorithm

ABSTRACT

In this paper, balance control of a bicycle robot is studied without either a trail or a mechanical regulator when the robot moves in an approximately rectilinear motion. Based on the principle of moment balance, an input nonaffine nonlinear dynamics model of the bicycle robot is established. A driving velocity condition is proposed to maintain the robot balance. The nonaffine nonlinear system is transformed into an affine nonlinear system by defining the equivalent control. Subsequently, a feedback linearization controller is designed for the equivalent control. We design a combined control algorithm of synchronous policy iteration based on the actor–critic architecture. The actor neural network (NN) is designed based on the feedback linearization control law. Weight tuning laws for the critic and actor NNs are proposed. The system closed-loop stability and convergence of the NN weights are guaranteed based on the Lyapunov analysis. The optimality of the equivalent control policy is guaranteed. To satisfy the driving velocity condition, the values of the steering angle and driving velocity are determined based on the optimal equivalent control. The effectiveness of the proposed algorithm is verified through simulations and real experiments.

1. Introduction

The bicycle robot, as a kind of vehicle that can assist in driving [1], has attracted significant attention. Bicycles are of great significance both in teaching robot design [2] and control theory [3]. Bicycles can also be used to illustrate various control concepts. A bicycle robot is a nonlinear system. Owing to its characteristics of static instability and dynamic stability, maintaining balance is the first requirement to consider.

Numerous studies have designed additional mechanical regulators, such as pendulums [4,5], reaction wheels [6–11], gyroscopes [12,13], legs [14] and a manipulator [15], for bicycle robots. The advantage of robots with mechanical regulators is that they can maintain balance at low speed or zero speed. However, bicycle robots with mechanical regulators have complex structures, large volumes, large weights, and high energy consumptions.

In addition, the dynamic characters of the bicycle are strongly affected by the trail. In general, the horizontal distance from the intersection of the extended line of the bicycle handlebar steering axis and the ground to where the front wheel touches the ground is called the trail or mechanical trail. The trail is an important index of bicycle handling characteristics. For a human rider, a higher trail makes the bicycle easier to ride with “no hands” and thus more stable [2]. Generally, bicycle robots can be classified into two classes according to whether they have a trail or not. Most existing studies have focused on bicycle robots that have a trail [4,5,7–10,12–20]. When a bicycle robot with a trail rotates the handlebar, it has decent directivity for the forward direction. Thus, its path control is easier than that without the trail. However, when turning the handlebar, it will encounter greater resistance than that encountered by a robot without a trail. It is obvious that the bicycle with the trail has better stability, while the bicycle without the trail has a smaller size and lower steering resistance. Therefore, bicycle robots without a trail should be studied [11,21]. We investigate a mechanical regulator-free bicycle robot without a trail in this paper. It has a simple structure, low energy consumption, and flexible turning. Meanwhile, its poor stability makes it more difficult to control than a bicycle that has a trail.

For a bicycle robot without a mechanical regulator, the balance of the robot can be maintained by controlling the steering angle or wheel speed. Most studies control the bicycle with constant speeds using the steering angle alone, e.g., [16,17,22]. Bicycle robots should be able to move at various speeds. This is more practical than moving at a constant speed. We admire the advantages of a bicycle with a

* Corresponding author.
E-mail addresses: guolei@bupt.edu.cn (L. Guo), linhy@bupt.edu.cn (H. Lin), jiangjiale@bupt.edu.cn (J. Jiang), songyuan@bupt.edu.cn (Y. Song), dgan@purdue.edu (D. Gan).

https://doi.org/10.1016/j.isatra.2023.11.032
Received 23 July 2023; Received in revised form 21 November 2023; Accepted 21 November 2023
Available online 23 November 2023
0019-0578/© 2023 ISA. Published by Elsevier Ltd. All rights reserved.

mechanical regulator that can maintain balance at low or zero speed. Meanwhile, we want to discard the mechanical regulators. If both the steering angle and driving speed can be used to simultaneously control the bicycle robot, a better control effect will be achieved. Therefore, we adopt the steering angle and wheel speed as the control inputs. Using different perspectives and modeling methods, such as Newton's Law [16], the Euler Lagrange equation [17,18], and Kane's equation [19], dynamics models with different complexity have been obtained. It is worth noting that their dynamics models are affine nonlinear systems. In retrospect, there is often a gap between dynamics modeling and controller design. It is difficult to design controllers using complex dynamics models [9]. Many studies have used approximation methods to linearize nonlinear dynamics models [21,23], such as the small-angle approximation or first-order Taylor expansion [10]. This significantly reduces the application range of the proposed control algorithm. In view of this, we propose a relatively simple dynamics model to maintain the stability of a bicycle robot by simultaneously adjusting both the steering angle and driving speed. Our model is a nonaffine nonlinear system. It is difficult for a mechanical regulator-free bicycle robot to maintain balance at very low speeds. Obviously, there is a lower bound on the forward driving speed of the robot. After the nonlinear system is approximately linearized, some approximate lower bounds are obtained [2,19]. However, the driving velocity condition was not further investigated in [2,19]. The driving velocity conditions required for a bicycle robot to maintain its balance remain an open problem. Considering this, we present a rigorous lower bound for the speed.

1.1. Related work

The model-based controller exhibits good real-time performance. Model-based methods such as feedback linearization (FBL) control [3,4,18,24] have been applied to bicycle robots. However, the FBL controller is not necessarily optimal. Moreover, tuning the controller parameters is resource intensive. To solve these problems, we introduce adaptive dynamic programming (ADP) to FBL control. ADP has been employed to address various optimal control problems [25–31]. It is also called reinforcement learning (RL) in the computational intelligence field. Most previous studies have focused on affine systems, such as [28–31]. For an affine nonlinear system, the optimal policy can be obtained by taking the derivative of the Lyapunov equation (LE) or Hamilton function. The inputs of the nonaffine nonlinear system are coupled in the input dynamics. For a nonaffine nonlinear system, the optimal policy cannot be solved as the affine nonlinear system. There are two approaches to solving this problem [32].

The first involves using the model-free ADP method to determine a policy that can minimize the Hamiltonian function [33]. Currently, there are numerous excellent model-free ADP algorithms based on neural network(NN) approximation techniques [28–30,33–35]. The advantage of the model-free ADP/RL algorithm is that it does not require system dynamics. Therefore, it can be used to design controllers for nonaffine nonlinear systems. However, this form of model-free algorithm requires high computational power to be used in real-time robot control. Moreover, energy consumptions will be high for the small mobile robots if model-free algorithms are used.

The second approach involves transforming the nonaffine nonlinear system into an equivalent affine nonlinear system. Evidently, actual control inputs are required to control the plant. Therefore, actual control inputs must be obtained based on the equivalent control. Existing method can only be applied when the inputs of a nonaffine nonlinear system are independent. In [36], the input dynamics of a nonaffine nonlinear system was divided into the product of two terms: one containing only the system state, and the other containing only the inputs. The nonlinear term of the inputs is regarded as the measure of control effort (i.e., equivalent control). Inspired by [36], we transform our nonaffine system into an affine system by introducing an equivalent control. However, the inputs of the nonaffine system studied in [36] are independent. More specifically, each input can only be expressed with the equivalent control. This is drastically different from our study. The inputs of the nonaffine nonlinear system that we study are not independent. Considering this, we propose a combined control algorithm.

Considering the characteristics of robots, we mainly study the optimal control problem for continuous-time (CT) systems. If the weight updates of the two NNs are not synchronized in the actor–critic based RL algorithm, a discrete-time (DT) weight controller will be brought into the actual CT dynamics system. Moreover, for the DT RL algorithm, where the weights of the critic and actor NNs are not tuned synchronously, it is obvious that there is still the problem of a slowly updating actor NN [37]. Some CT ADP/RL algorithms utilize the gradient descent method to synchronously tune the parameters of both critic and actor NNs [38–41]. This is termed synchronous RL (SRL). SRL does not require an initial admissible policy. This can solve the problem of controller parameter tuning. In engineering practice, tuning the parameters of a nonlinear controller requires significant effort. Moreover, the performance of the controller is not necessarily admissible or optimal when the manually selected weights are used. Therefore, this problem can be solved by using SRL for robot control. There are two methods for designing an actor NN. One is based on the analytical expression of the optimal control policy derived from the value function to design an actor NN [42]. The other is to construct actor NN using subjective or algorithm-selected neurons [43]. Both methods need to determine the type and number of neurons used in the critic NN or actor NN. Usually, it is necessary to subjectively set a sufficient number of neurons to ensure the approximation accuracy. When the actor NN has a high dimension, the computational burden will be huge [11,44]. Moreover, the stability analysis is not sufficient in [11,44]. Considering this, we design the actor NN based on the FBL control law and make it approximate the optimal control policy via SRL. The weights of the two NNs are continuously updated. Accordingly, we can tune the FBL controller adaptively and make it optimal. In addition, we can guarantee the convergence of the critic and actor NNs weights using a Lyapunov analysis.

1.2. Contributions

In this article, a combined balancing control scheme is studied for a bicycle robot nonaffine nonlinear system by combining both FBL control and SRL techniques. Compared with previous related works, the main contributions of this paper are as follows.

1. Considering the steering angle and wheel driving velocity as control inputs, a nonaffine nonlinear dynamics model of a bicycle robot without a trail is established. A rigorous condition for the driving velocity is presented to maintain the balance of the robot. This condition describes the lower bound of the driving speed more rigorously and accurately than approximation methods [2,19].

2. A combined control algorithm based on SRL is proposed for a nonaffine nonlinear system with dependent control inputs. Compared with [36], solving the equations with real control inputs as variables is not required. The control law for the driving velocity is obtained to satisfy the driving velocity condition. We realize a method of setting the driving velocity first and then solving the steering angle corresponding to the optimal equivalent control using the SRL process. The control problem of nonaffine nonlinear systems with dependent control inputs is solved. Our study enriches the control methods for nonaffine robotic systems.

3. The actor NN is designed based on the FBL control law. Using our method, the resulting actor NN is more compact and interpretable than other design methods [42–44]. The weight tuning laws of both critic and actor NNs are presented. The FBL control law approximates the optimal equivalent control by tuning its parameters. The resource intensive work of manually tuning the controller parameters is avoided. Based on the Lyapunov analysis, the closed-loop stability of the system and convergence of the NNs weights are proved.

1.3. Structures and notations

The rest of this paper is organized as follows. In Section 2, a nonaffine nonlinear system dynamics model for a bicycle robot is established. The nonaffine system is transformed into an affine system by defining the equivalent control. The driving velocity condition for the robot to maintain balance when it moves forward is derived. In Section 3, a combined control algorithm based on SRL is proposed. In Section 4, the actor and critic NNs are parameterized, and the weight tuning laws are derived. Section 5 presents the simulation verification. The experimental results for a real robot are presented in Section 6, and conclusions are given in Section 7.

We use $\|x\|$ to denote the Euclidean norm $\sqrt{x^\top x}$ of the vector x. $\lambda_{max}(X)$ and $\lambda_{min}(X)$ denote the maximum and minimum eigenvalues of the square matrix X, respectively. For ease of description, the equations may omit the variable; for example, f denotes $f(x)$. $\nabla f(x) = \partial f/\partial x$ is the gradient of the function f.

2. Dynamics model and balance control principle analysis of a bicycle robot

2.1. The bicycle robot dynamics model

In Fig. 1(a), O_1 and O_2 indicate the center of the rear and front wheels, respectively. l denotes the distance O_1O_2. P_A indicates the contact point between the rear wheel of the bicycle and the ground, and P_B represents that of the front wheel. B^* denotes the center of mass (COM) of the bicycle. When the robot roll angle is zero, the height of the robot COM is h. B_1^* and B_2^* denote the point on the front wheel fork axis and the rear wheel fork axis, respectively. Moreover, $B_1^*P_A = B_2^*P_B = h$. When the steering angle is adjusted to a certain angle, the front wheel axis and the rear wheel axis are no longer parallel. The two axes intersect at a point P_C because they have the same wheel radius. Let line P_CP_D be the vertical line of the plane $O_1O_2P_C$. The line P_CP_D is the instantaneous rotation axis of the robot.

Fig. 1(b) presents a top view of the bicycle robot when the robot roll angle is zero. Here, l_1 indicates the horizontal distance of O_1B^*, v_f indicates the velocity of the front wheel COM. Additionally, R_w denotes the radius of both the front and rear wheels, r_0 corresponds to the turning radius of the robot COM, r_1 denotes the turning radius of the rear wheel, and r_2 corresponds to the turning radius of the front wheel. Moreover, q_1 denotes the frame yaw angle, q_2 symbolizes the roll angle, q_3 represents the steering angle of the front wheel around axis $O_2B_2^*$, and q_4 corresponds to the rotation angle of the front wheel around axis O_2P_C. $\dot{q}_i(i = 1, 2, \dots, 4)$ is the derivative of the variable with respect to time. Finally, $a^{(n)}$ indicates the centrifugal acceleration of B^*.

It can be observed from Fig. 1(b) that the turning radius of the COM of the rear wheel satisfies the following equation:

$$r_1 = l\cot q_3. \tag{1}$$

Similarly, the turning radius of the front wheel COM satisfies the following equation:

$$r_2 = \frac{l}{\sin q_3}. \tag{2}$$

The turning radius of the COM of the bicycle satisfies the following equation:

$$r_0 = \sqrt{{l_1}^2 + {r_1}^2} = \sqrt{{l_1}^2 + l^2\cot^2 q_3}. \tag{3}$$

Based on the assumption that there is no relative sliding between the wheel and ground, the COM velocity v_f of the front wheel satisfies the following equation:

$$v_f = R_w\dot{q}_4. \tag{4}$$

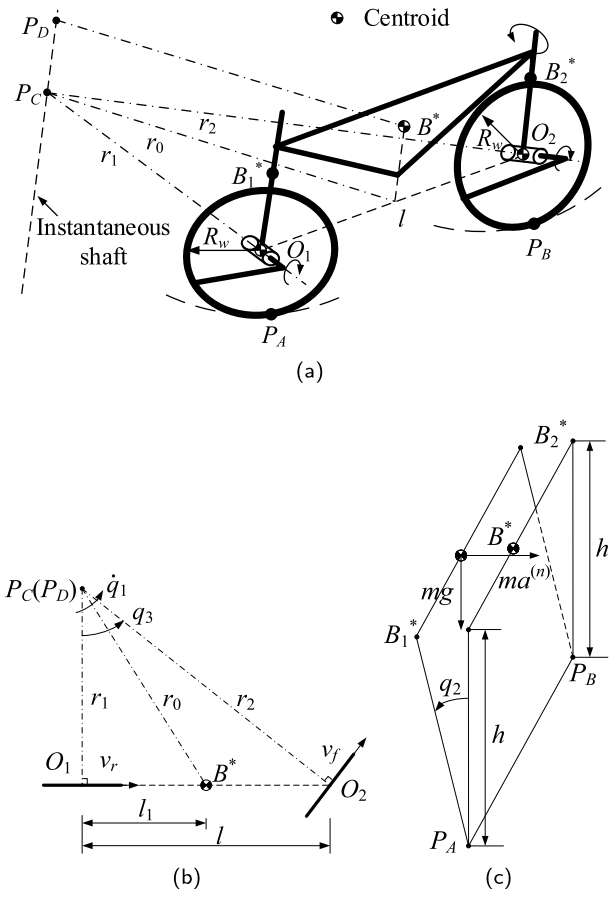


Fig. 1. Bicycle robot. (a) Axonometric view. (b) Top view. (c) Rear view.

In addition, the frame yaw angular velocity $\dot{q}_1$ can be derived to satisfy the following equation:

$$\dot{q}_1 = \frac{v_f}{r_2} = \frac{R_w\sin q_3}{l}\dot{q}_4. \tag{5}$$

At this moment, the bicycle body has a circular motion. The centripetal acceleration is $a^{(n)} = \dot{q}_1^2 r_0$. Substituting (3) and (5) into this equation, we obtain

$$a^{(n)} = \left((\frac{R_w}{l})^2\sin q_3\sqrt{{l_1}^2\sin^2 q_3 + l^2\cos^2 q_3}\right)\dot{q}_4^2. \tag{6}$$

The force analysis of the robot is shown in Fig. 1(c). It can be observed that the bicycle robot will topple when subjected to the moment of gravity. As shown in Fig. 1(b), q_2 is zero when the robot is in the equilibrium position. We let the counterclockwise direction be positive. The mechanical model of the system can be written as:

$$J\ddot{q}_2 - mgh\sin q_2 + ma^{(n)}h\cos q_2 = 0. \tag{7}$$

Here, $J = mh^2$. Substituting (6) into (7), we obtain

$$h\ddot{q}_2 - g\sin q_2 + \tag{8}$$
$$\cos q_2\left((\frac{R_w}{l})^2\sin q_3\sqrt{{l_1}^2\sin^2 q_3 + l^2\cos^2 q_3}\right)\dot{q}_4^2 = 0.$$

For (8), let $q_2 = x_1$, $\dot{q}_2 = x_2$ and $q_3 = u_1$, $\dot{q}_4 = u_2$. We then obtain the following nonaffine nonlinear system:

$$\dot{x}(t) = f(x(t)) + \overline{g}(x(t), u_1(t), u_2(t)), \tag{9}$$

where $x(t) \in \mathbb{R}^2$ is the state vector $[x_1, x_2]^\top$, $u_1 \in \mathbb{R}$, and $u_2 \in \mathbb{R}$ denote the control inputs. For ease of description, we omit the variable t and simplify $x(t)$ to x. $f(x) = [f_1(x), f_2(x)]^\top = [f_1, f_2]^\top = [x_2, g\sin x_1/h]^\top$

indicates the drift dynamics. $\bar{g}(x,u_1,u_2) = [0,-\cos x_1 p(u_1){u_2}^2]^\top$ is the input dynamics, where

$$p(u_1) = \frac{1}{h}(\frac{R_w}{l})^2 \sin u_1 \sqrt{{l_1}^2 \sin^2 u_1 + l^2 \cos^2 u_1}.$$

2.2. Balance control principle analysis

The equivalent control is defined as follows:

$$u = p(u_1){u_2}^2 \triangleq \frac{a^{(n)}}{h}. \tag{10}$$

The following affine nonlinear system can be obtained:

$$\dot{x}(t) = f(x(t)) + g^*(x(t))u(t), \tag{11}$$

where $g^*(x) = [g_1^*(x), g_2^*(x)]^\top = [g_1^*, g_2^*]^\top = [0, -\cos x_1]^\top$. It is clear that when $q_2 = x_1$ is close to 90 deg, the bicycle robot falls to the ground. At this time, adjustments to the robot are no longer valid. The domain of $q_2 = x_1$ should be $[-30, 30]$ deg. Thus, $\cos q_2 = \cos x_1 = -g_2^* \neq 0$. We can use FBL to design the control law by letting $\ddot{q}_2 = \ddot{x}_1 = \dot{x}_2 = v$. The control law u can be deduced via (11) as follows:

$$u = \frac{v - f_2}{g_2^*}, \tag{12}$$

where v is the virtual control. Let x_{1d} be the reference signal. Here, e denotes the tracking error, i.e., $e = x_1 - x_{1d}$. Accordingly, we obtain $\dot{e} = x_2 - \dot{x}_{1d}$ and $\ddot{e} = \dot{x}_2 - \ddot{x}_{1d}$. Let the virtual control be $v = \ddot{x}_{1d} - k_2\dot{e} - k_1 e$. The error dynamics model is $\ddot{e} + k_2\dot{e} + k_1 e = 0$. The coefficients k_1 and k_2 should be positive to ensure convergence of the error dynamics system. To allow the robot to maintain balance, let the reference signal be zero, i.e., $x_{1d} = 0$. This is crucial when the robot moves in an approximately rectilinear motion. The subsequent derivations of the proposed algorithm are based on this setting. Accordingly, the virtual control is derived as:

$$v = -k_2 x_2 - k_1 x_1. \tag{13}$$

Substituting (10) and (13) into (12) yields

$$u = \frac{-k_2 x_2 - k_1 x_1 - f_2}{g_2^*} = \tag{14}$$
$$\frac{1}{h}\Big((\frac{R_w}{l})^2 \sin u_1 \sqrt{{l_1}^2 \sin^2 u_1 + l^2 \cos^2 u_1}\Big){u_2}^2.$$

If the driving wheel speed is zero, the control input (10) is zero. There will be no input to achieve balance control, and the robot will naturally fall. Consider the forward direction of the bicycle robot as the positive direction, i.e. $\dot{q}_4 = u_2 > 0$. Under this condition, (14) can be identically transformed into

$$\frac{h^2(-k_2 x_2 - k_1 x_1 - f_2)^2}{(\frac{R_w}{l})^4 {u_2}^4 {g_2^*}^2} = \tag{15}$$
$$\sin^2 u_1\big(({l_1}^2 - l^2)\sin^2 u_1 + l^2\big).$$

To simplify the expression, set

$$C = \frac{h^2(-k_2 x_2 - k_1 x_1 - f_2)^2}{(\frac{R_w}{l})^4 {u_2}^4 {g_2^*}^2} = \frac{h^2 u^2}{(\frac{R_w}{l})^4 {u_2}^4}, \tag{16}$$

$$\lambda = \sin^2 u_1. \tag{17}$$

The steering wheel must swing to allow the robot to maintain balance. Thus, u_1 can be zero, λ may be zero. It is obvious that $C \geqslant 0, 1 > \lambda \geqslant 0$. By substituting (16) and (17) into (15), this can be represented as

$$({l_1}^2 - l^2)\lambda^2 + l^2\lambda - C = 0. \tag{18}$$

Eq. (18) can be considered to be in the form $a\lambda^2 + b\lambda + c = 0$, where $a = {l_1}^2 - l^2, b = l^2, c = -C$. It is obvious that $l > l_1 > 0$. We have the relationship $a < 0, b > 0, c \leqslant 0$.

When $\Delta \geqslant 0$, (18) has real solutions. When $\Delta = 0$, there are two identical positive roots $\lambda_0 = l^2(l^2 - {l_1}^2)^{-1}/2$. The meaning of the roots is that the steering angle corresponds to a constant nonzero angle. When $\Delta = 0$, the system is open-loop. When $\Delta > 0$, there are two different roots. Therefore, (18) must satisfy the following conditions:

$$\begin{cases} \Delta = l^4 + 4({l_1}^2 - l^2)C > 0 \\ 2 > -\frac{b}{a} = -\frac{l^2}{{l_1}^2 - l^2} > 0 \\ 1 > \frac{c}{a} = \frac{-C}{{l_1}^2 - l^2} \geqslant 0 \end{cases}. \tag{19}$$

According to the relationship $l > l_1$ and the definition of C, it is obvious that $-\frac{b}{a} = \frac{l^2}{l^2 - {l_1}^2} > 0$ and $\frac{c}{a} = \frac{C}{l^2 - {l_1}^2} \geqslant 0$. The second inequality of (19) shows the requirement of the structural design of the bicycle, which is $l^2 > 2{l_1}^2$. This is the requirement for the geometric parameters of the robot to achieve balance. The first and third inequalities of (19) are represented as given below

$$T_0 = \min\{\frac{l^4}{4(l^2 - {l_1}^2)}, l^2 - {l_1}^2\} > C \geqslant 0.$$

When C satisfies the inequality $T_0 > C$, two distinct roots can be obtained for (18).

$$\lambda_{1,2} = \frac{l^2 \pm \sqrt{l^4 - 4(l^2 - {l_1}^2)C}}{2(l^2 - {l_1}^2)}. \tag{20}$$

It is evident that $\lambda_2 < \lambda_0 < \lambda_1$. The solution λ_1 is always greater than zero and will never equal zero. λ_2 can be equal to zero. This conforms to the physical meaning that the steering angle oscillates near zero when the robot moves with a rectilinear motion. When performing a rectilinear motion, the solution λ_2 is adopted.

To simplify the formula, we define

$$D = \frac{l}{R_w}\sqrt[4]{\frac{h^2(-k_2 x_2 - k_1 x_1 - f_2)^2}{T_0 {g_2^*}^2}}. \tag{21}$$

Since $u_2 > 0$. Substituting (16) into $T_0 > C$, the following inequality can be obtained:

$$u_2 > D. \tag{22}$$

Inequality (22) is a crucial condition for maintaining the robot balance. According to (17), we can obtain the steering angle u_1 with the root λ_2 in (20):

$$u_1 = \pm \arcsin\sqrt{\frac{l^2 - \sqrt{l^4 - 4(l^2 - {l_1}^2)C}}{2(l^2 - {l_1}^2)}}. \tag{23}$$

By substituting (16) $C = h^2 l^4 u^2/(R_W u_2)^4$ into (23), we obtain the equation for calculating u_1 using u_2:

$$u_1 = \pm \arcsin\Big(l\sqrt{\frac{1 - \sqrt{1 - 4(l^2 - {l_1}^2)\frac{h^2 u^2}{{R_W}^4 {u_2}^4}}}{2(l^2 - {l_1}^2)}}\Big),\ u_2 > D.$$

This relationship indicates that the system balance can be achieved by controlling the steering angle u_1 or front-wheel driving speed u_2. It is also possible to adopt a method that simultaneously controls both u_1 and u_2. However, there is a strong coupling relationship between the steering angle and the wheel driving speed. Therefore, we can determine only one before obtaining the other. To address this problem and achieve a better control effect, we use the method proposed in the next section.

3. Combined control algorithm based on synchronous reinforcement learning

In this section, we propose the method to control the robot with both u_1 and u_2 simultaneously. According to (22), u_2 is considered to be greater than its lower bound. When the system state approaches

the equilibrium point, the lower limit of u_2 is close to zero. It is well known that when the bicycle has a rectilinear motion, the system will be unstable if the speed is too low. Therefore, the driving speed is determined to ensure that u_2 is not too low. Accordingly, u_2 is calculated as follows:

$$\begin{cases} u_2 = \alpha D + (1-\alpha)u_2(0) + \beta & D > u_2(0) \\ u_2 = u_2(0) + \beta & D \leqslant u_2(0) \end{cases}, \tag{24}$$

where $\alpha > 1$ and $0 < \beta < (\alpha - 1)u_2(0)$ are the constants. $u_2(0) > 0$ indicates the expected initial driving speed. It is difficult to change the speed suddenly in engineering practice, and the piecewise function (24) designed is continuous.

We use the following analysis to verify that (24) can satisfy the velocity constraint $u_2 > D$. When $D > u_2(0)$, using the condition $\alpha > 1$, we obtain the following relation: $\alpha D + (1-\alpha)u_2(0) > D$. Using the condition $\beta > 0$, we can obtain $u_2 = \alpha D + (1-\alpha)u_2(0) + \beta > D + \beta > D$. When $D \leqslant u_2(0)$, using the condition $\beta > 0$, we can obtain $u_2 = u_2(0) + \beta > D$. In theory, as long as β satisfies $\beta > 0$, it can satisfy the requirements of the above derivation process. Although we could set u_2 to a relatively big value theoretically. However, in engineering practice, achieving extremely high driving speeds is difficult. The constraint $\beta < (\alpha - 1)u_2(0)$ acts to limit the speed to as small a value as possible. The condition $D > u_2(0)$ can act as an early warning. When $D > u_2(0)$, the robot might become unstable. According to (24), the control law $u_2 = \alpha D + (1-\alpha)u_2(0) + \beta$ can be adopted to ensure that the condition $u_2 > D$ holds at this time. Therefore, the velocity condition of the robot is ensured.

We can solve the problem of tuning the parameters k_1 and k_2 by determining the optimal equivalent control. We obtain the optimal equivalent control u based on SRL. For system (11), we investigate the problem of minimizing the following value function for any $t > 0$ as:

$$V(x) = \int_t^\infty r(x(\tau), u(\tau))\mathrm{d}\tau, \tag{25}$$

where $r(x,u) \triangleq Q(x) + Ru^2 \geq 0$ for a positive definite function $Q(x)$ and a positive number R. The Lyapunov equation (LE) of nonlinear system (11), i.e., the infinitesimal version of (25), is proposed as follows:

$$r(x,u) + \nabla V^\top(x)\big(f(x) + g^*(x)u\big) = 0. \tag{26}$$

We use $V^*(x) = \min V(x)$ to denote the optimal value function. The optimal policy that can minimize (25) is as follows:

$$u^*(x(t)) = \arg\min_u \int_0^\infty r(x(\tau), u(\tau))\mathrm{d}\tau. \tag{27}$$

The optimal policy can be determined by minimizing the following Hamiltonian function:

$$H(x, u, \nabla V(x)) = r(x,u) + \nabla V^\top(x)\big(f(x) + g^*(x)u\big). \tag{28}$$

The optimal policy can be obtained by substituting the function $r(x,u) = Q(x) + Ru^2$ into (26) and taking the derivative of u:

$$u^*(x(t)) = -\frac{1}{2}R^{-1}g^{*\top}(x)\nabla V^*(x). \tag{29}$$

Substituting (29) into (26),we obtain the Hamilton–Jacobi-Bellman (HJB) equation:

$$Q(x) + \nabla V^{*\top}(x)f(x) - \frac{1}{4}\nabla V^{*\top}(x)g^*(x)R^{-1}g^{*\top}(x)\nabla V^*(x) = 0. \tag{30}$$

For the affine nonlinear system (11), the equivalent control (i.e., the acceleration control) (14) based on FBL indicates that we can calculate u in real time. However, the u obtained in this way is not necessarily optimal. Since (14) and (29) describe the same control variable u, we can determine k_1 and k_2 by setting (14) equal to (29). The ideal parameters k_1 and k_2 are unknown, and we need to gradually approximate them using NNs. Subsequently, the obtained k_1 and k_2 can make the control law (14) the optimal control that can minimize the performance index (25). We can obtain that

$$u = \frac{-k_2x_2 - k_1x_1 - f_2}{g_2^*} = u^*(x(t)). \tag{31}$$

In [36], nonaffine systems in which real inputs are independent were studied. That is, a one-to-one correspondence exists between each real input and the equivalent control in [36]. Each real control input in [36] can be represented uniquely by the equivalent control u. However, our real control inputs, u_1 and u_2 are coupled nonlinearly. This differs from the results reported in [36]. In our nonaffine system, the real inputs are dependent. Our goal is to determine the values of the two actual inputs based on the optimal equivalent control u^*.

Eq. (23) shows that u_1 can be obtained using the variable C. We can obtain specific values of C based on u^* and u_2. We obtain the optimal equivalent control u^* via (29). Under the condition that u^* is known, we can determine the value of only one real input before determining the value of another. u_2 can be obtained via (24). Therefore, we have u_2, which satisfies condition (22). By substituting (31) into (16), we have $C = h^2l^4u^{*2}/(R_Wu_2)^4$. Based on u_2, we obtain C. Based on C, u_1 is obtained using (23). Compared with [36], solving the equations with real control inputs as variables is not required. In the next section, we describe the implementation of the above calculation process using an SRL-based algorithm.

4. Approximate optimal solution based on NNs

Even though the dynamics model of (11) is completely known, it is still difficult to solve the HJB equation (30). Currently, the general method is to use neural networks to obtain the approximate solution of the HJB equation. A method for implementing the proposed combined control algorithm using NNs is discussed. To solve the HJB equation (30), the value function should be approximated. According to the Weierstrass high-order approximation theorem, there exists an NN such that $V(x)$ and its gradient $\nabla V(x)$ are uniformly approximated. We can obtain the following NN

$$V(x) = W_c^\top\phi_c(x) + \varepsilon_c(x), \tag{32}$$

$$\nabla V(x) = \big(\nabla\phi_c(x)\big)^\top W_c + \nabla\varepsilon_c(x), \tag{33}$$

where $W_c \in \mathbb{R}^{N_c}$ denotes the ideal weight vector. $\phi_c(x) \in \mathbb{R}^{N_c}$ is the critic NN activation function. N_c is the number of neurons, $\phi_c(x) = [\phi_{c1}(x), \ldots, \phi_{cN_c}(x)]^\top$. Here, $\varepsilon_c(x)$ indicates the critic NN approximation error.

We can obtain the approximation error of (26) as

$$r(x,u^*) + \nabla V^{*\top}(x)\big(f(x) + g^*(x)u^*\big) = \varepsilon_H, \tag{34}$$

where ε_H indicates the approximate residual of the critic NN. It can be represented as $\varepsilon_H = -\nabla\varepsilon_c^\top(x)\big(f(x) + g^*(x)u^*\big)$.

Since the ideal weight vector in (32) is unknown, we need to gradually approximate it with the NN. The value function estimation is represented as

$$\hat{V}(x) = \big(\hat{W}_c(t)\big)^\top\phi_c(x). \tag{35}$$

For ease of description, we omit the variable t and simplify $\hat{W}_c(t)$ to $\hat{W}_c$, and the estimation of the Hamiltonian function is deduced as

$$\hat{H}(x, u, \nabla\phi_c{}^\top\hat{W}_c) = r(x,u) + \hat{W}_c^\top\nabla\phi_c\big(f(x) + g^*(x)u\big). \tag{36}$$

Since $H(x, u^*, \nabla V^*) = 0$, by using the Hamiltonian approximation error, we define the objective function as

$$E_c = \frac{1}{2}\big(\hat{H}(x, u, \nabla\phi_c{}^\top\hat{W}_c)\big)^2. \tag{37}$$

We use the gradient descent method to tune the critic NN weights so that E_c can be minimized as

$$\dot{\hat{W}}_c = -\eta_c\frac{\partial E_c}{\partial\hat{W}_c} = -\eta_c\frac{\sigma}{(\sigma^\top\sigma + 1)^2}\big(\hat{W}_c^\top\sigma + r(x,u)\big), \tag{38}$$

where η_c indicates the learning rate of the critic NN, $\sigma = \nabla\phi_c\big(f(x) + g^*(x)u\big)$. To guarantee that the critic NN weights can converge, we need to ensure the persistence of the excitation (PE) condition [38]. Define the signal $\overline{\sigma}$ as

$$\overline{\sigma} = \frac{\sigma}{\sigma^\top\sigma + 1}. \tag{39}$$

Definition 1. The PE condition: At any given time, the signal $\overline{\sigma}$ is persistently excited over the interval $[t, t+T]$ if there exist constants $\beta_1 > 0$ and $\beta_2 > 0$, such that

$$\beta_1 I \leqslant \int_t^{t+T} \overline{\sigma}(\tau)\overline{\sigma}^\top(\tau)\mathrm{d}\tau \leqslant \beta_2 I.$$

Theorem 1. *This is theorem 1 defined in [38]. Let the residual error (34) be bounded. For any admissible policy, the tuning law (38) can make the approximation error of the critic NN weights converge exponentially under the condition of PE.*

We can design the following NN based on (14) such that the policy u can be uniformly approximated:

$$u = W_a^\top\phi_a(x) - \frac{f_2}{g_2^*} + \varepsilon_a(x), \tag{40}$$

where $W_a = [k_1, k_2]^\top$ represents the ideal weight vector. Additionally, $\varepsilon_a(x)$ indicates the actor NN approximation error. $\phi_a(x) \in \mathbb{R}^2$ is the actor NN activation function, i.e., $\phi_a(x) = [-x_1/g_2^*, -x_2/g_2^*]^\top = [x_1/\cos x_1, x_2/\cos x_1]^\top$.

When the robot maintains balance, $x_1 \in [-30, 30]$ deg, so the basis vector $\phi_a(x)$ is bounded. Let $\hat{k}_1$ and $\hat{k}_2$ be estimations of ideal parameters k_1 and k_2 that can make (31) true. We denote $\hat{W}_a(t) = [\hat{k}_1, \hat{k}_2]^\top$. For ease of description, we omit the variable t and simplify $\hat{W}_a(t)$ to $\hat{W}_a$. In the learning process of the actor NN, as the weight estimation $\hat{W}_a$ of the actor NN tends toward the ideal weight W_a, the control policy tends toward (40). The parameters $\hat{k}_1$ and $\hat{k}_2$ in the control law tend toward the ideal values k_1 and k_2, respectively. Thus, the control law that optimizes the performance index $V(x)$ can be obtained, and the parameters of the control law can be tuned automatically.

Remark 1. The actor NN (40) is designed based on the FBL control law. Accordingly, we achieve the following two goals. The first is to optimize the FBL control law based on the SRL. The second is to avoid the workload of manually tuning the parameters in engineering practice.

Assumption 1. For a given compact set $\Omega \in \mathbb{R}^n$:

a. $f(\cdot)$ and $g^*(\cdot)$ are Lipschitz and $g^*(\cdot)$ is bounded by a constant $\|f(x)\| \leqslant b_f\|x\|$, $\|g^*(x)\| \leqslant b_g$.

b. The reconstruction error of the NNs and the gradient of the value function NN error are bounded so that $\|\varepsilon_c\| < \varepsilon_{cm}$, $\|\varepsilon_a\| < \varepsilon_{am}$, $\|\nabla\varepsilon_c\| < \varepsilon_{dm}$.

c. The activation functions of the NNs and the gradients of the value function NN activation functions are bounded so that $\|\phi_c(x)\| < \phi_{cm}$, $\|\phi_a(x)\| < \phi_{am}$, $\|\nabla\phi_c(x)\| < \phi_{dm}$.

d. The ideal weights of the NNs are bounded so that $\|W_c\| < W_{cm}$, $\|W_a\| < W_{am}$.

These are the standard assumptions in nonlinear approximation and learning control [38,45]. These assumptions are needed in the proof. System (11) satisfies Assumption 1.a. Since the bound of the approximation error can be arbitrarily small according to the Weierstrass high-order approximation theorem [46], Assumption 1.b is practical. The activation functions of both critic and actor NNs used in this paper are bounded. Assumption 1.c is satisfied. Assumption 1.d is conventional. It ensures the boundedness of the weights while tuning the weights of two NNs.

The estimation of the policy by the actor NN is

$$\hat{u} = \hat{W}_a^\top\phi_a(x) + \frac{f_2}{\cos x_1}. \tag{41}$$

The estimation of the optimal policy by the critic NN is

$$\hat{u}^* = -\frac{1}{2}R^{-1}g^{*\top}(x)\nabla\phi_c{}^\top(x)\hat{W}_c. \tag{42}$$

The approximation error of the actor NN can be obtained by subtracting (42) from (41). To find the weight tuning law of $\hat{W}_a$, the objective function is defined as

$$E_a = \frac{1}{2}(\hat{u} - \hat{u}^*)^2. \tag{43}$$

The weight tuning law of the actor NN is

$$\dot{\hat{W}}_a = \overbrace{-\eta_a(-\phi_a\hat{W}_c^\top B\hat{W}_a)}^{1}\overbrace{-\eta_a(Y\hat{W}_a)}^{2} \overbrace{-\eta_a\tanh\big((\hat{W}_a^\top\phi_a + \frac{f_2}{\cos x_1} + \frac{1}{2}R^{-1}g^{*\top}\nabla\phi_c{}^\top\hat{W}_c)\phi_a\big)}^{3}, \tag{44}$$

where η_a indicates the learning rate of the actor NN and Y denotes a positive parameter. B is shown as follows:

$$B = \frac{\sigma}{(\sigma^\top\sigma + 1)^2}R\phi_a{}^\top. \tag{45}$$

The first term of (44) is intended to eliminate the unstable term in the Lyapunov function. The second term is also added to ensure stability. The third term is used to reduce E_a. The hyperbolic tangent function is used to ensure that the gradient is bounded. The approximation errors are defined as $\tilde{W}_c = W_c - \hat{W}_c$, and $\tilde{W}_a = W_a - \hat{W}_a$.

Theorem 2. *Let Assumption 1 and the PE condition be satisfied and the parameters be selected appropriately. The existence of the admissible policy is guaranteed. Let the tuning laws of the critic and actor NN be provided by (38) and (44), respectively. The closed-loop system state, the critic NN weight estimation error $\tilde{W}_c$, and the actor NN weight estimation error $\tilde{W}_a$ are uniformly ultimately bounded (UUB).*

Proof. See Appendix.

The estimation of the policy $\hat{u}$ tends toward the optimal policy when tuning the weight $\hat{W}_a$. Based on the estimation of the policy, D is obtained using (21). u_2 can be obtained according to (24) to control the driving speed. Based on the estimation of the policy and u_2, C is obtained using (16). u_1 can be obtained to control the steering angle using (23). The pseudocode for the combined control algorithm is presented in Algorithm 1. The second step is to employ the ordinary differential equation (ode) solver to solve the system closed-loop dynamics $\dot{x} = f(x) + g^*(x)\hat{u}$ in a single time step. A schematic of the control system is shown in Fig. 2.

Algorithm 1 Combined control algorithm based on SRL

1: Start with initial state $x(0)$ and initial weights $\hat{W}_c(0)$,$\hat{W}_a(0)$. Determine the initial speed using (24). Compute the initial steering angle using (23).
2: Propagate t,$x(t)$.
3: Update $\hat{W}_c(t)$ using (38). Compute $\hat{V}(t)$ using (35).
4: Update $\hat{W}_a(t)$ using (44).
5: Compute $\hat{u}(t)$ using (41).
6: Compute D using (21). Compute $u_2(t)$ using (24). Compute C using (16). Compute $u_1(t)$ using (23).
7: Repeat 2–6 until $\|\hat{W}_c(t) - \hat{W}_c(t-1)\| < 10^{-4}$ and $\|\hat{W}_a(t) - \hat{W}_a(t-1)\| < 10^{-4}$. Then repeat step 6.

Our combined control is realized using a model-based synchronous RL algorithm that can be adopted for an online control system, as

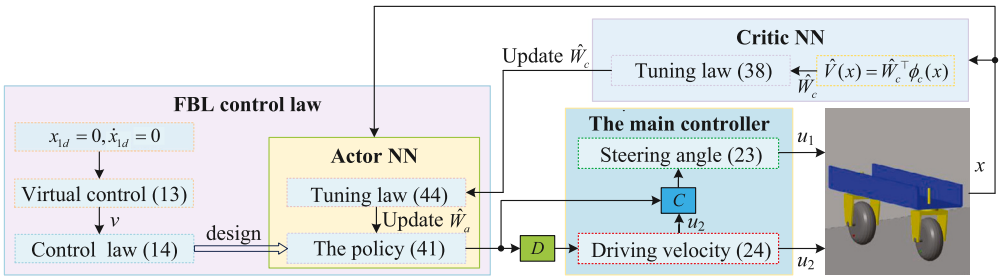


Fig. 2. Schematic of control system.

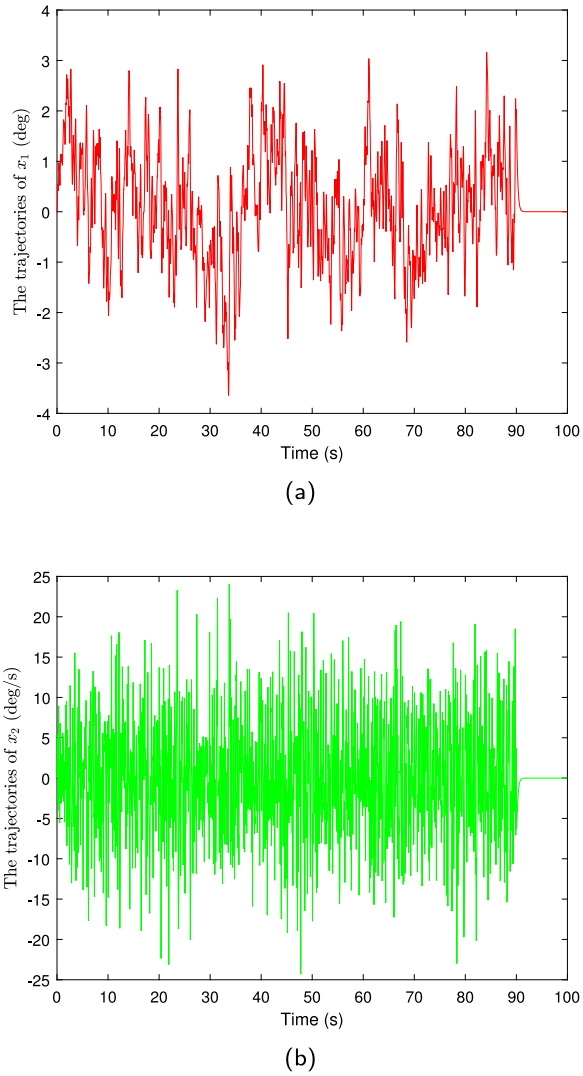


Fig. 3. Trajectories of the system state based on SRL. (a) x_1. (b) x_2.

presented in Algorithm 1 and Fig. 2. Under Assumption 1 and the weight-tuning law (44) of the actor NN obtained via Lyapunov analysis, the uniformly ultimately bounded (UUB) property of the system is guaranteed.

5. Simulation

The parameters are set as: $l = 0.38$ m, $l_1 = 0.19$ m, $h = 0.15$ m, $R_w = 0.054$ m, $m = 8.35$ kg. To calculate the value function, we use $r(x,u) = Q(x) + Ru^2$, where $R = 0.01$ and $Q(x) = x_1{}^2 + x_2{}^2$. Since the cost-to-go $r(x,u)$ and value function $V(x)$ are user-defined, the activation functions of the critic NN are manually set based on experience. We set the basis vector $\phi_c(x) = [x_1{}^2, x_1x_2, x_2{}^2]^\top$. The nodes of the actor NN are designed based on the FBL control law. That is, we design the actor NN using (41). In this way, the parameters of the FBL controller can be adaptively tuned using SRL algorithm. The initial state is $x(0) = [0,0]^\top$ and the initial weights are $\hat{W}_c(0) = [10,10,10]^\top$, $\hat{W}_a(0) = [10,10]^\top$. The learning rates are set as $\eta_c = 50000$ and $\eta_a = 300$. We have adopted the following two measures to treat the PE problem. On one hand, we use the superposition of 100 randomly generated sinusoidal signals to ensure that the excitation signal frequency is rich enough. On the other hand, we employ a 90-second excitation duration to sufficiently excite the system state. The excitation signal (disturbance signal) $0.5\sum_{i=1}^{100}\sin(\omega_i t)$ is added to $\hat{u}$ in the first 90 s of simulation. The simulation stops at $t_f = 100$ s. We set $\alpha = 1.1$ and $\beta = 0.1$, $u_2(0) = 20.944$ rad/s in (24).

The trajectories of x_1 and x_2 obtained using Algorithm 1 are shown in Fig. 3(a) and Fig. 3(b), respectively. The fluctuation range of roll angle x_1 is $[-3.6413, 3.1618]$ deg. The fluctuation range of roll angular velocity x_2 is $[-24.3031, 24.0481]$ deg/s. It means that the bicycle robot remains stable during training. As shown in Fig. 3, due to the presence of disturbance signal within the initial 90 s, the system state trajectory oscillates and does not converge to zero. According to the definitions of the basis vectors of critic and actor NNs, the output of each neuron in these two NNs is not always zero in the first 90 s. This indicates that persistent excitation is present during this period. The excitation signal prevents the system state from converging to the origin during the excitation period, ensuring that the weights of the actor and critic NNs converge to their ideal values, respectively. After the disturbance is stopped, the state can be stabilized to the origin quickly. Approximately 1 s after the disturbance stops, the roll angle and roll angular velocity converge to the equilibrium point, showing that the proposed algorithm can ensure the bicycle balance.

The critic and actor NNs are trained online. As shown in Figs. 4(a) and 4(b), the weights of the critic NN converge at about 10 s, and those of the actor NN converge at about 40 s. Subsequently, after 90 s of training, all the weights of the two NNs converge to $\hat{W}_c(t_f) = [12.5994, 2.6316, 0.1907]^\top$, $\hat{W}_a(t_f) = [66.0067, 19.0252]^\top = W_{a(SRL)}$.

Remark 2. After about 10 s, the disturbance signal has no effect on the weights of the critic NN. Similarly, the disturbance signal has no effect on the weights of the actor NN after about 40 s. The weights of both the critic and actor NNs actually converged at approximately 40 s. The excitation lasts for 90 s to ensure that the state is fully excited. The excitation after 40 s dose not cause any weights change.

According to (17) and $\sin^2 u_1 = \lambda_2 < \lambda_0$, there is a relationship $|u_1| < \arcsin\sqrt{\lambda_0} = 54.7356$ deg exists when we use (23) to obtain the steering angle. In Fig. 5(a), the steering angle u_1 fluctuates in the range $[-32.0986, 34.5180]$ deg. The fluctuation is within the range of (23). The random disturbance signal is added to the control input during the first 90 s of the simulation. The steering angle u_1, as illustrated in Fig. 5(a), is adjusted accordingly under the random disturbance.

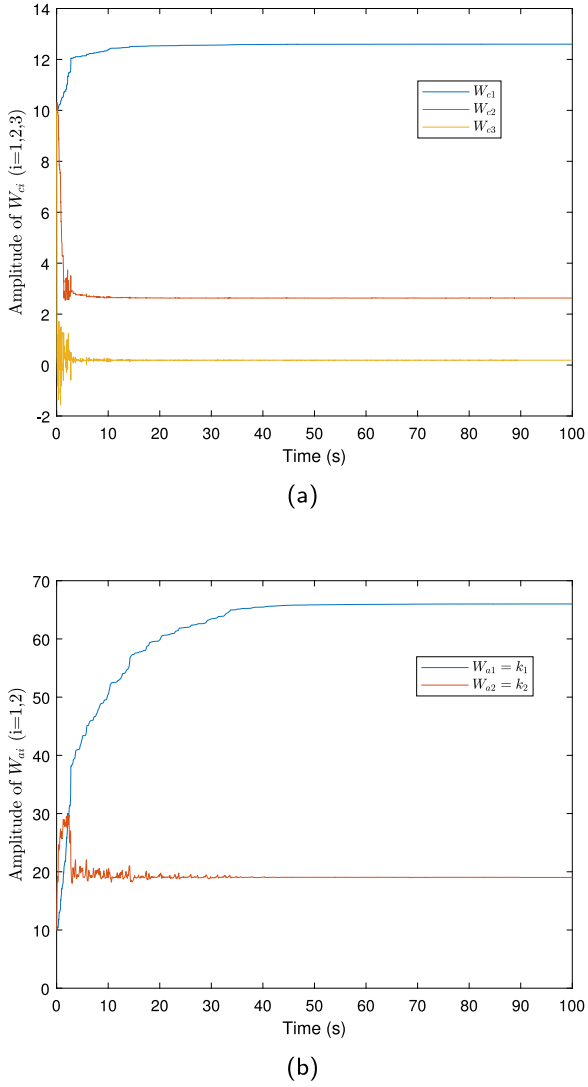


Fig. 4. Evolution of the NN weights of SRL. (a) The critic. (b) The actor.

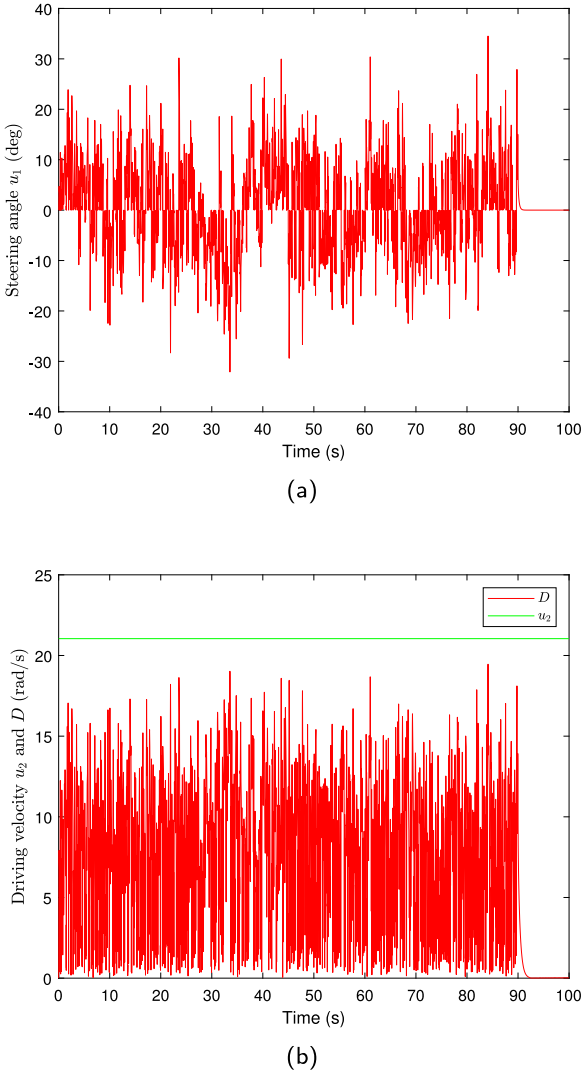


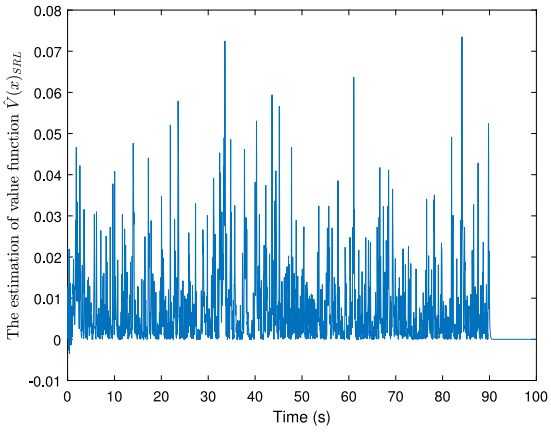
Fig. 5. Curves of the real control based on SRL. (a) u_1. (b) u_2 and D.

Fig. 5(b) shows that the green curve u_2 is always above the red curve D, which satisfies the speed condition.

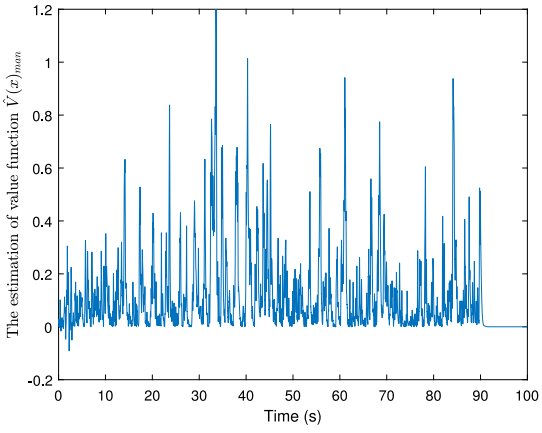
To illustrate the significance of the proposed algorithm, we conducted a simulation experiment of FBL controller with manually selected parameters. To ensure a fair comparison with the SRL controller, which adaptively adjusts the parameters of FBL controller, we established the following two experimental settings. Firstly, the FBL controller uses the same critic NN as the SRL controller. Therefore, we can evaluate the performance index of FBL controllers using $\hat{V}(x)$, similar to the evaluation mechanism employed for SRL controllers. Secondly, there is the same disturbance signal in the FBL controller simulation experiment as in the SRL controller simulation experiment. In contrast to the SRL controllers, the manually selected parameters $k_1 = 15$ and $k_2 = 5$ in the FBL controller remain constant. Consequently, the FBL controller lacks adaptability. For comparison, the value function estimations of the SRL controller and FBL controller are shown in Fig. 6(a) and Fig. 6(b), respectively. Our comparison of experimental results focuses on two aspects: system state convergence and performance index.

From the perspective of system state convergence, the results indicate that both the SRL controller and FBL controller effectively drive the system state to zero rapidly when the disturbance signal is turned off at 90 s. In terms of performance indices, Fig. 6(a) reveals that the peak of the estimated value function $\hat{V}(x)_{SRL}$ is 0.0735. As depicted in Fig. 6(b), the maximum value of $\hat{V}(x)_{man}$ is 1.1994. Comparative experiments reveal that the maximum value of $\hat{V}(x)_{man}$ for the FBL controller is approximately 16.3 times that of the SRL controller. As the value function $V(x)$ reflects the loss of the control system, it becomes evident that the performance indicators achieved by the FBL controller with manually determined parameters are not as good as those obtained by the SRL controller.

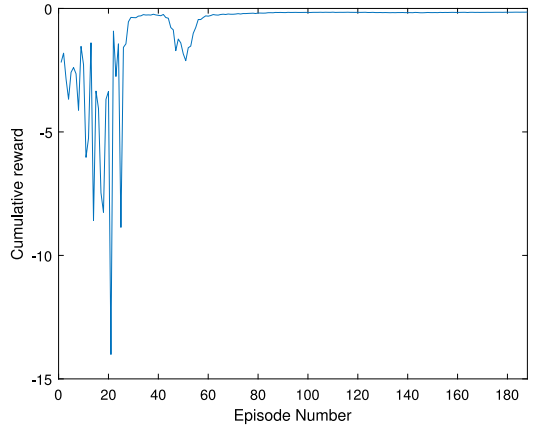
We use the twin delayed deep deterministic policy gradient algorithm (TD3) [37] to replace the SRL module in the combined control algorithm for comparison. It is worth noting that SRL differs from TD3 in three main ways. First, the goal of the SRL algorithm is to minimize the cumulative cost-to-go $r(x,u)$. However, the goal of the TD3 algorithm is to maximize the cumulative reward. Since minimizing the cumulative $r(x,u)$ of the quadratic form is equivalent to maximizing the cumulative $-r(x,u)$, we set the reward of TD3 to $-r(x,u)$. SRL calculates the value function $V(x)$ by integrating the cost-to-go over time. The TD3 algorithm obtains the cumulative reward of each episode by summing the instantaneous reward of each time step. To facilitate comparison, we adopted the method of integrating $-r(x,u)$ over time to calculate the cumulative reward in the simulation based on the TD3 algorithm. The TD3-based algorithm uses the same actor NN as the SRL-based algorithm. Second, the critic NN used in the TD3 algorithm is



(a)



(b)



(c)

Fig. 6. Comparison of performance index. (a) SRL. (b) man. (c) TD3.

different from that of SRL. The critic NN of the TD3 algorithm uses more layers and more neurons in each layer than the SRL algorithm. The TD3-based algorithm uses a deep NN with 2273 learnable parameters. The critic NN used in the SRL-based algorithm is a shallow NN with 3 learnable parameters. Therefore, the TD3 algorithm illustrates the learning effect by improving the reward trajectory. This is one reason why our algorithm is faster than the TD3 algorithm. Third, we guarantee the UUB property of the SRL algorithm via the Lyapunov

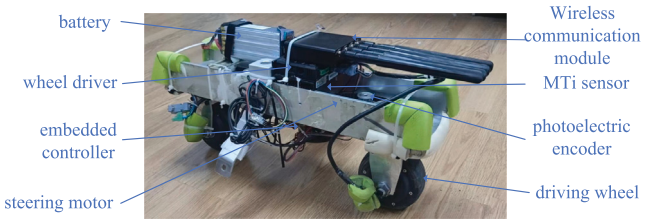


Fig. 7. Our real bicycle robot.

analysis. However, the TD3 algorithm does not have the UUB property guaranteed by the Lyapunov analysis. This is another reason why our algorithm is better than the TD3 algorithm. In the simulation, the sampling time is set to 0.01 s, and the time length of each episode is 5 s. The evolution process of its cumulative reward is shown in Fig. 6(c). At approximately the 180th episode (900 s), the cumulative reward of TD3 algorithm reaches its maximum value. At this time, the weight of actor NN is $W_{a(TD3)} = [63.5371, 19.4023]^\top$.

The algorithm based on TD3 converges at approximately 900 s. The maximum cumulative reward of the TD3-based algorithm is −0.1465. When the PE ends at 90 s, the value function of SRL rapidly converges to its optimal value of zero. Comparing Figs. 6(a) and 6(c), the performance of the algorithm based on SRL is better than that of the algorithm based on TD3.

6. Experiments based on the robot prototype

In this section, we use the real bicycle robot depicted in Fig. 7 to verify the effectiveness of the proposed control algorithm. The robot parameters are the same as those in the simulation. The front wheel of the robot is the driving wheel. The steering motor controls the handlebar angle through a synchronous belt drive. A photoelectric encoder is used to measure the steering angle. An MTi sensor is used to measure the roll and yaw angles.

For the hub motor that drives the robot forward, we adopt the speed control mode. The driving velocity is obtained using Algorithm 1. We adopt the position control mode for the motor that rotates the handlebar. The steering angle is obtained using Algorithm 1. It is worth mentioning that the proposed algorithm can run online. However, owing to the limited computing capacity of the STM32F427 controller used in our robot, the experiments we carried out were performed offline based on the optimal parameters obtained from the simulations. That is, we consider $W_{a(SRL)} = [k_1, k_2]^\top = [66.0067, 19.0252]^\top$. Using the optimal weights obtained via simulation, the sixth step of Algorithm 1 is repeated in the offline version of Algorithm 1.

The real robot starts to move from a static state and maintains balance while moving forward. The screenshots of the experimental videos are shown in Fig. 8.

According to Fig. 9(a), the steering angle has a maximum amplitude of $u_1 = -9.533$ deg at the initial time. When the roll angle of the robot converges to nearly 0 deg after 0.7 s, the steering angle fluctuates around zero. The fluctuation range is $[-6.792, 4.418]$ deg.

Remark 3. Since the robot starts to move from a static state, it is inevitable that $u_2 < D$ will hold in the initial stage and last for a very short time. In Fig. 9(b), within approximately 0.1 s, the angular velocity u_2 is less than D. However, the adopted $u_2(0) = 20.944$ rad/s is greater than D, so we can still use (23) to calculate the theoretical value of u_1 to which the front wheel should be rotated at this time. Considering the rapid rise of u_2, the green curve is consistently above the red curve after 0.1 s.

The yaw angle curve of the robot is shown in Fig. 9(c). The maximum yaw angle amplitude is 7.7464 deg. Eventually, the yaw angle

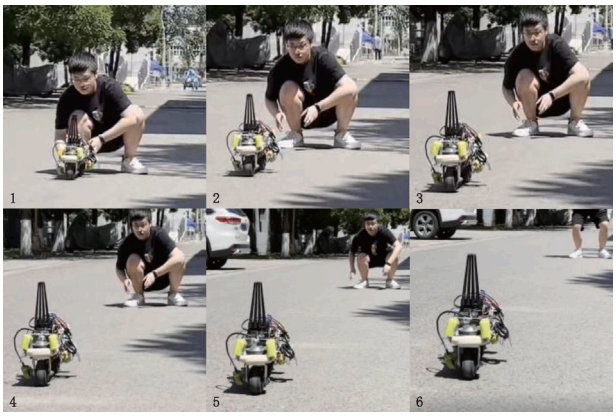


Fig. 8. Video screenshots of the real robot experiment.

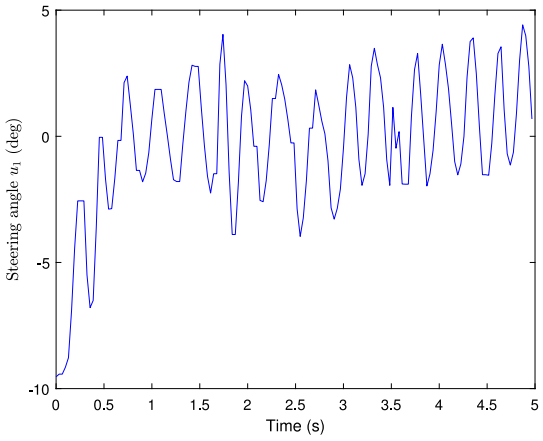
is −2.7273 deg. This indicates that the bicycle robot can track an approximately straight line by adjusting the steering angle under variable driving velocities.

To clearly show the roll angle curve, the unit of roll angle is set as deg and the unit of roll angular velocity is set as rad/s in Fig. 10(a). The robot starts to move from the initial roll angle of approximately 1.5 deg. The roll angle converges to nearly 0 deg within approximately 0.7 s. This shows that the robot can maintain balance when it is moving forward. The roll angle fluctuates around 0 deg owing to external disturbances, such as uneven pavement.

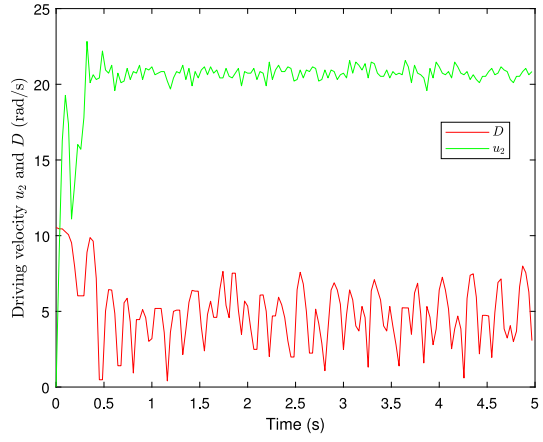
Remark 4. In computer simulations, the system output can be quickly stabilized at the origin after the disturbance signal is stopped. However, several disturbances can occur in the real world. We cannot turn off the disturbance signals in actual experiments as in the simulation. For example, asphalt pavements exhibit different friction coefficients and flatness in different areas. When the robot moves on the ground with different roughness and flatness, the frictions and collisions between the wheels and the ground differ significantly. This interferes with the control of bicycle robots in actual experiments. In summary, owing to the difference between the simulation and actual experiments, the system output did not converge to zero in actual experiments. It oscillated slightly near its origin.

A comparison experiment was performed based on the manually selected weight $W_{a(man)} = [15, 5]^{\top}$ at the same site. The results are shown in Fig. 10(b). The absolute value of the initial roll angle in Fig. 10(a) is almost twice as that in Fig. 10(b). Nevertheless, we can observe that when using the manually selected parameters, the fluctuation range of the roll angle is larger than that when using the parameters obtained via SRL.

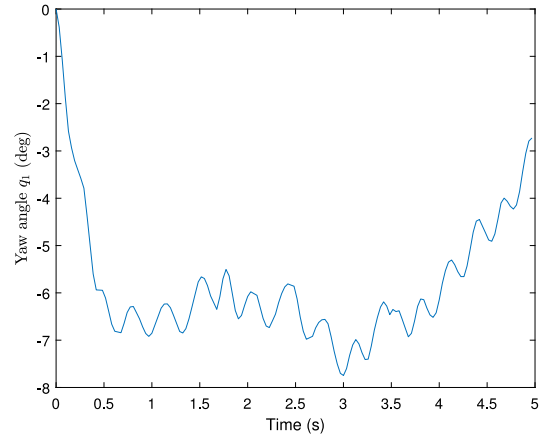
For further comparison, we use the weight $W_{a(TD3)}$ to carry out experiments at the same experimental site. The results are shown in Fig. 10(c). The absolute value of the initial roll angle in Fig. 10(a) is almost seven times that in Fig. 10(c). Even so, the swing amplitude of the roll angle when using $W_{a(SRL)}$ is smaller than that when using $W_{a(TD3)}$. The comparative experiments show that the fluctuation frequency is smaller when using $W_{a(TD3)}$ or $W_{a(man)}$ than when using $W_{a(SRL)}$. This means that the controller using $W_{a(SRL)}$ is more sensitive than the other two. The results show that the bicycle robot can maintain balance when moving forward. It is verified that we can effectively control the nonaffine nonlinear system (9). In the experiments, we can effectively ensure that condition (22) is met.



(a)



(b)

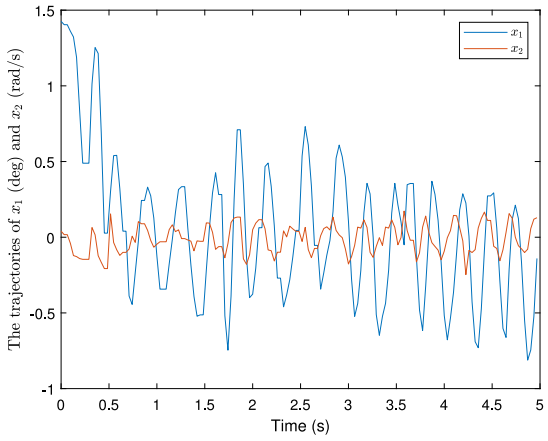


(c)

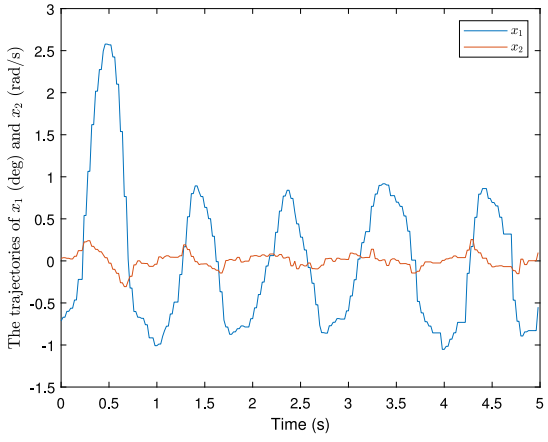
Fig. 9. Curves of the robot using $W_{a(SRL)}$. (a) u_1. (b) u_2 and D. (c) Yaw angle q_1.

7. Conclusions and future work

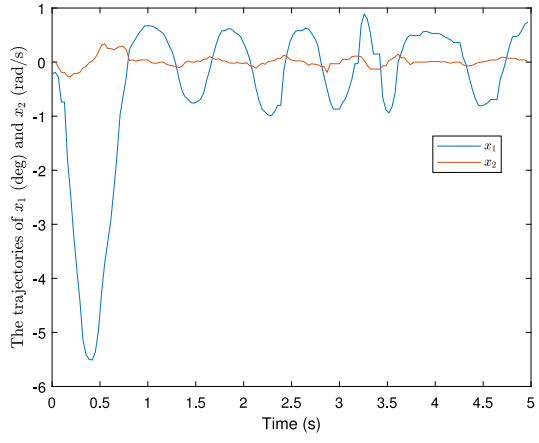
A bicycle robot without a mechanical regulator and a trail is investigated in this paper. The robot dynamics model is established. The condition of the wheel driving velocity is proposed to maintain the robot balance. The nonaffine nonlinear system is transformed into an



(a)



(b)



(c)

Fig. 10. Trajectories of x_1 and x_2. (a) $W_{a(SRL)}$. (b) $W_{a(man)}$. (c) $W_{a(TD3)}$.

affine nonlinear system by defining the equivalent control. The FBL control law for equivalent control is designed. The actor NN is designed based on the FBL control law. The weight tuning laws of the critic and actor NNs are provided. The UUB property of the system is guaranteed based on the Lyapunov analysis. A combined control algorithm is designed based on the SRL. We created a prototype of the bicycle robot, the effectiveness of the proposed algorithm was experimentally verified.

The algorithm proposed in this paper can only be used when the robot moves in an approximately rectilinear motion. Although the proposed method cannot be directly applied to achieve circular motion, it can be improved to achieve circular motion. It is necessary to realize the circular motion of bicycle robot based on this paper. The study in this paper can be considered a prerequisite step toward achieving circular motion. The tracking signal $x_{1d} = 0$ can be changed to an appropriate nonzero value to achieve circular motion. If we aim to ensure that the trajectory of robot forms a perfect circle, the radius of this circle is a fixed value r_a. According to (3), the steering angle $|u_1|$ should also be a nonzero constant. At this point, we have $p(u_1) \neq 0$. Therefore, the expression of the wheel driving speed can be solved using (15). In this way, an improved algorithm can be proposed to achieve strict circular motion. If we aim to approximate the trajectory of robot to a circle, the radius r_0 is a variable. Let δ denotes a small positive value, we have $r_0 \in [r_a, r_a + \delta]$. The range of u_1 can be determined using (3). Depending on the nonzero x_{1d}, the steering angle u_1 should be appropriately calculated using either λ_1 or λ_2 based on the different turning radius. The steering angle should fall within a small neighborhood of a fixed nonzero value. Consequently, an improved algorithm can be proposed to achieve approximate circular motion. However, when conducting research on circular motion based on the above ideas, it is also necessary to consider the switching problem of control laws or the relationship between the velocity control and velocity constraint. Additionally, the design of the adaptive optimal parameter tuning law for the controller in circular motion and its stability proof need to be studied. A model-based SRL algorithm was used in the study. Information on the drift dynamics $f(x)$ and input dynamics $g^*(x)$ is required. In future research, model-free SRL algorithms with low computational power will be studied for application to the real-time control of mobile robots.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported by the National Natural Science Foundation of China under Grant 61105103.

Appendix

Proof of Theorem 2. In some equations, the variable is dropped for ease of description, e.g. f denotes $f(x)$. We define a Lyapunov function

$$\begin{aligned} L &= V(x) + L_1 + L_2 \\ &= V(x) + \frac{1}{2}\tilde{W}_c^{\top}\eta_c^{-1}\tilde{W}_c + \frac{1}{2}\tilde{W}_a^{\top}\eta_a^{-1}\tilde{W}_a. \end{aligned} \tag{46}$$

The derivative of (46) with respect to time t is

$$\dot{L} = \dot{V}(x) + \dot{L}_1 + \dot{L}_2, \tag{47}$$

where

$$\dot{L}_1 = \tilde{W}_c^{\top}\eta_c^{-1}\dot{\tilde{W}}_c, \tag{48}$$

$$\dot{L}_2 = \tilde{W}_a^{\top}\eta_a^{-1}\dot{\tilde{W}}_a. \tag{49}$$

Substituting (41) into the first term $\dot{V}(x)$ of (47), it is derived as

$$\begin{aligned} \dot{V}(x) &= W_c^{\top}\nabla\phi_c\left(f(x) + g^*(x)\hat{u}\right) + \nabla\varepsilon_c^{\top}\left(f(x) + g^*(x)\hat{u}\right) \\ &= W_c^{\top}\sigma_1 - W_c^{\top}\nabla\phi_c g^*(x)\tilde{W}_a^{\top}\phi_a + \varepsilon_1. \end{aligned} \tag{50}$$

Here,

$$\varepsilon_1 = \nabla\varepsilon_c{}^{\top}\big(f(x)+g^*(x)(\hat{W}_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1})\big), \tag{51}$$

$$\sigma_1 = \nabla\phi_c\big(f(x)+g^*(x)(W_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1})\big). \tag{52}$$

According to HJB equation (30), we can obtain

$$W_c{}^{\top}\sigma_1 = -Q(x)-R(W_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1})^2+\varepsilon_{HJB}(x). \tag{53}$$

Under Assumption 1, ε_{HJB} is bounded [38]; i.e., $\|\varepsilon_{HJB}\| < b_\varepsilon$. Substituting (53) into (50), we can obtain

$$\begin{aligned}\dot{V}(x) = &-Q(x)-R(W_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1})^2-\\ &W_c{}^{\top}\nabla\phi_c g^*(x)\tilde{W}_a{}^{\top}\phi_a+\varepsilon_1+\varepsilon_{HJB}(x).\end{aligned} \tag{54}$$

During the process of weight updating,

$$\sigma = \nabla\phi_c\big(f(x)+g^*(x)\hat{u}\big). \tag{55}$$

To facilitate the subsequent derivation, we define

$$\overline{R}_1 = R(W_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1})^2, \tag{56}$$

$$\overline{R}_2 = R(\hat{W}_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1})^2. \tag{57}$$

The derivative of $\tilde{W}_c = W_c - \hat{W}_c$ with respect to time t is obtained by substituting (38) into it and combining (39):

$$\begin{aligned}\dot{\tilde{W}}_c = \eta_c\frac{\overline{\sigma}}{m_s}\Big(&\hat{W}_c{}^{\top}\nabla\phi_c\big(f(x)+g^*(x)\hat{u}\big)+\overline{R}_2-\overline{R}_1-\\ &W_c{}^{\top}\nabla\phi_c\big(f(x)+g^*(x)(W_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1})\big)+\varepsilon_{HJB}\Big),\end{aligned} \tag{58}$$

where $m_s = \sigma^{\top}\sigma+1$. Substituting (56) and (57) into (58), one obtains

$$\begin{aligned}\dot{\tilde{W}}_c = \eta_c\frac{\overline{\sigma}}{m_s}\Big(&-\tilde{W}_c{}^{\top}\sigma+\varepsilon_{HJB}-W_c{}^{\top}\nabla\phi_c g^*(x)\tilde{W}_a{}^{\top}\phi_a+\\ &\phi_a{}^{\top}\hat{W}_a R\hat{W}_a{}^{\top}\phi_a-\phi_a{}^{\top}W_a RW_a{}^{\top}\phi_a-2R\tilde{W}_a{}^{\top}\phi_a\frac{f_2}{\cos x_1}\Big).\end{aligned} \tag{59}$$

Substituting (33) and (40) into (29), we can obtain

$$\begin{aligned}W_c{}^{\top}\nabla\phi_c g^*(x) = &-2\phi_a{}^{\top}W_a R-2\varepsilon_a{}^{\top}R-\\ &\nabla\varepsilon_c{}^{\top}g^*(x)-2R\frac{f_2}{\cos x_1}.\end{aligned} \tag{60}$$

Substituting (60) into (59), we can obtain

$$\begin{aligned}\dot{\tilde{W}}_c = \eta_c\frac{\overline{\sigma}}{m_s}\Big(&-\tilde{W}_c{}^{\top}\sigma+\phi_a{}^{\top}\tilde{W}_a R\tilde{W}_a{}^{\top}\phi_a+2\varepsilon_a{}^{\top}R\tilde{W}_a{}^{\top}\phi_a+\\ &\nabla\varepsilon_c{}^{\top}g^*(x)\tilde{W}_a{}^{\top}\phi_a+\varepsilon_{HJB}\Big).\end{aligned} \tag{61}$$

Substituting (61) into (48) yields

$$\begin{aligned}\dot{L}_1 = \tilde{W}_c{}^{\top}\frac{\overline{\sigma}}{m_s}\Big(&-\tilde{W}_c{}^{\top}\sigma+2\varepsilon_a{}^{\top}R\tilde{W}_a{}^{\top}\phi_a+\\ &\nabla\varepsilon_c{}^{\top}g^*(x)\tilde{W}_a{}^{\top}\phi_a+\varepsilon_{HJB}\Big)+\tilde{W}_c{}^{\top}\frac{\overline{\sigma}}{m_s}\phi_a{}^{\top}\tilde{W}_a R\tilde{W}_a{}^{\top}\phi_a.\end{aligned} \tag{62}$$

According (45), the last term in (62) is derived as

$$\begin{aligned}&\tilde{W}_c{}^{\top}\frac{\overline{\sigma}}{m_s}\phi_a{}^{\top}\tilde{W}_a R\tilde{W}_a{}^{\top}\phi_a = \tilde{W}_a{}^{\top}\phi_a\hat{W}_c{}^{\top}B\hat{W}_a+\\ &\tilde{W}_a{}^{\top}\phi_a(-W_c{}^{\top}BW_a+\tilde{W}_c{}^{\top}BW_a+W_c{}^{\top}B\tilde{W}_a).\end{aligned} \tag{63}$$

Subsequently, (62) is derived as

$$\begin{aligned}&\dot{L}_1 = \tilde{W}_c{}^{\top}\frac{\overline{\sigma}}{m_s}(-\tilde{W}_c{}^{\top}\sigma+\varepsilon_{HJB})+\\ &\tilde{W}_c{}^{\top}\frac{\sigma}{(\sigma^{\top}\sigma+1)^2}\big(2\varepsilon_a{}^{\top}R+\nabla\varepsilon_c{}^{\top}g^*(x)\big)\tilde{W}_a{}^{\top}\phi_a+\\ &\tilde{W}_a{}^{\top}\phi_a\hat{W}_c{}^{\top}B\hat{W}_a+\tilde{W}_a{}^{\top}\phi_a(-W_c{}^{\top}BW_a+W_c{}^{\top}B\tilde{W}_a)+\\ &\tilde{W}_a{}^{\top}\phi_a R\tilde{W}_c{}^{\top}\frac{\sigma}{(\sigma^{\top}\sigma+1)^2}\phi_a{}^{\top}W_a.\end{aligned} \tag{64}$$

For the second term and the last term of (64), Young's inequality is used as follows:

$$\begin{aligned}&\tilde{W}_c{}^{\top}\frac{\sigma}{(\sigma^{\top}\sigma+1)^2}\big(2\varepsilon_a{}^{\top}R+\nabla\varepsilon_c{}^{\top}g^*(x)\big)\tilde{W}_a{}^{\top}\phi_a\leqslant\\ &\frac{1}{2\varepsilon}\tilde{W}_c{}^{\top}\frac{\sigma\sigma^{\top}}{(\sigma^{\top}\sigma+1)^2}\tilde{W}_c+\frac{\varepsilon}{2(\sigma^{\top}\sigma+1)^2}\phi_a{}^{\top}\tilde{W}_a M^2\tilde{W}_a{}^{\top}\phi_a,\end{aligned} \tag{65}$$

$$\begin{aligned}&\tilde{W}_a{}^{\top}\phi_a R\tilde{W}_c{}^{\top}\frac{\sigma}{(\sigma^{\top}\sigma+1)^2}\phi_a{}^{\top}W_a\leqslant\\ &\frac{\varepsilon}{2(\sigma^{\top}\sigma+1)^2}\phi_a{}^{\top}\tilde{W}_a R^2\tilde{W}_a{}^{\top}\phi_a+\\ &\frac{1}{2\varepsilon}\tilde{W}_c{}^{\top}\frac{\sigma}{\sigma^{\top}\sigma+1}\phi_a{}^{\top}W_a W_a{}^{\top}\phi_a\frac{\sigma^{\top}}{\sigma^{\top}\sigma+1}\tilde{W}_c.\end{aligned} \tag{66}$$

Here, $M = 2\varepsilon_a{}^{\top}R+\nabla\varepsilon_c{}^{\top}g^*(x)$. Using (65) and (66), (64) becomes

$$\begin{aligned}&\dot{L}_1\leqslant(-1+\frac{1}{2\varepsilon}+\frac{1}{2\varepsilon}\phi_a{}^{\top}W_a W_a{}^{\top}\phi_a)\tilde{W}_c{}^{\top}\frac{\sigma\sigma^{\top}}{(\sigma^{\top}\sigma+1)^2}\tilde{W}_c+\\ &\tilde{W}_c{}^{\top}\frac{\sigma}{(\sigma^{\top}\sigma+1)^2}\varepsilon_{HJB}+\tilde{W}_a{}^{\top}\phi_a(-W_c{}^{\top}BW_a+W_c{}^{\top}B\tilde{W}_a)+\\ &\frac{\varepsilon}{2(\sigma^{\top}\sigma+1)^2}\phi_a{}^{\top}\tilde{W}_a(M^2+R^2)\tilde{W}_a{}^{\top}\phi_a+\tilde{W}_a{}^{\top}\phi_a\hat{W}_c{}^{\top}B\hat{W}_a.\end{aligned} \tag{67}$$

Substituting (44) into (49) yields

$$\begin{aligned}&\dot{L}_2 = \tilde{W}_a{}^{\top}(-\phi_a\hat{W}_c{}^{\top}B\hat{W}_a)+\tilde{W}_a{}^{\top}YW_a-\tilde{W}_a{}^{\top}Y\tilde{W}_a+\\ &\tilde{W}_a{}^{\top}\tanh\big((\hat{W}_a{}^{\top}\phi_a+\frac{f_2}{\cos x_1}+\frac{1}{2}R^{-1}g^{*\top}\nabla\phi_c{}^{\top}\hat{W}_c)\phi_a\big).\end{aligned} \tag{68}$$

Set $Q(x) = x^{\top}qx$. As mentioned before, $\|\varepsilon_{HJB}\| < b_\varepsilon$. According to Assumption 1, the bound of (54) is given as

$$\begin{aligned}&\dot{V}(x)\leqslant-\lambda_{min}(q)\|x\|^2+R\big(W_{am}\phi_{am}+max(\frac{f_2}{\cos x_1})\big)^2+\\ &W_{cm}\phi_{dm}b_g\phi_{am}\|\tilde{W}_a\|+\varepsilon_{dm}b_f\|x\|+\varepsilon_{dm}b_g W_{am}\phi_{am}+\\ &\varepsilon_{dm}b_g\|\tilde{W}_a\|\phi_{am}+\varepsilon_{dm}max(f_2)+b_\varepsilon.\end{aligned} \tag{69}$$

Here, $\lambda_{min}(q)$ denotes the minimum eigenvalue of matrix q. According to $\|\overline{\sigma}\| = \|\frac{\sigma}{\sigma^{\top}\sigma+1}\| < 1$ and Assumption 1, (67) is derived as

$$\begin{aligned}&\dot{L}_1\leqslant(-1+\frac{1}{2\varepsilon}+\frac{1}{2\varepsilon}\phi_{am}{}^2W_{am}{}^2)\frac{\|\tilde{W}_c{}^{\top}\sigma\|^2}{(\sigma^{\top}\sigma+1)^2}+\\ &b_\varepsilon\frac{\|\tilde{W}_c{}^{\top}\sigma\|}{(\sigma^{\top}\sigma+1)^2}+\frac{\varepsilon}{2(\sigma^{\top}\sigma+1)^2}\phi_a{}^{\top}\tilde{W}_a(M^2+R^2)\tilde{W}_a{}^{\top}\phi_a+\\ &\phi_{am}^2W_{cm}W_{am}R\|\tilde{W}_a\|+\tilde{W}_a{}^{\top}\phi_a W_{cm}R\phi_a{}^{\top}\tilde{W}_a+\\ &\tilde{W}_a{}^{\top}\phi_a\hat{W}_c{}^{\top}B\hat{W}_a.\end{aligned} \tag{70}$$

According to Assumption 1 and theorem of spectral radius, (68) is derived as

$$\begin{aligned}&\dot{L}_2\leqslant\sqrt{N_a}\|\tilde{W}_a\|+W_{am}\big(\lambda_{max}(Y)+\varepsilon_Y\big)\|\tilde{W}_a\|\\ &-\tilde{W}_a{}^{\top}Y\tilde{W}_a-\tilde{W}_a{}^{\top}\phi_a\hat{W}_c{}^{\top}B\hat{W}_a.\end{aligned} \tag{71}$$

N_a is the number of neurons in the actor NN, and ε_Y is a positive number. Adding (69), (70) and (71), we have

$$\begin{aligned}&\dot{L} = \dot{V}(x)+\dot{L}_1+\dot{L}_2\leqslant\|x\|\big(-\lambda_{min}(q)\|x\|+\varepsilon_{dm}b_f\big)+c\\ &+\|\tilde{W}_c{}^{\top}\overline{\sigma}\|(-X\|\tilde{W}_c{}^{\top}\overline{\sigma}\|+\frac{b_\varepsilon}{\sigma^{\top}\sigma+1})\\ &+\|\tilde{W}_a\|\big(-\lambda_{min}(T_1)\|\tilde{W}_a\|+d\big).\end{aligned} \tag{72}$$

Here,

$$X = 1-\frac{1}{2\varepsilon}-\frac{1}{2\varepsilon}\phi_{am}{}^2W_{am}{}^2, \tag{73}$$

$$T_1 = Y-\frac{\varepsilon}{2(\sigma^{\top}\sigma+1)^2}\phi_a(M^2+R^2)\phi_a{}^{\top}-\phi_a W_{cm}R\phi_a{}^{\top}, \tag{74}$$

$$\begin{aligned}c = &R\big(W_{am}\phi_{am}+max(\frac{f_2}{\cos x_1})\big)^2+\varepsilon_{dm}b_g W_{am}\phi_{am}\\ &+\varepsilon_{dm}max(f_2)+b_\varepsilon,\end{aligned} \tag{75}$$

$$\begin{aligned}d = &W_{cm}\phi_{dm}b_g\phi_{am}+\varepsilon_{dm}b_g\phi_{am}+\phi_{am}^2W_{cm}W_{am}R\\ &+\sqrt{N_a}+W_{am}\big(\lambda_{max}(Y)+\varepsilon_Y\big).\end{aligned} \tag{76}$$

Select the parameters ϵ and Y appropriately so that both X and T_1 are positive definite. When the following conditions are met, $\dot{L}$ is negative.

$$\|x\| > \frac{\epsilon_{dm} b_f}{2\lambda_{min}(q)} + \sqrt{\frac{(\epsilon_{dm} b_f)^2}{4\lambda_{min}^2(q)} + \frac{c}{\lambda_{min}(q)}}, \tag{77}$$

$$X\|\tilde{W}_c^{\top}\overline{\sigma}\| > \frac{b_\epsilon}{\sigma^{\top}\sigma + 1}, \tag{78}$$

$$\lambda_{min}(T_1)\|\tilde{W}_a\| > d. \tag{79}$$

According to the Lyapunov analysis and Remark 1, both the state and the weights are ultimately uniformly bounded. □

References

[1] Jeon W, Xie Z, Craig C, Achtemeier J, Alexander L, Morris N, et al. A smart bicycle that protects itself: Active sensing and estimation for car-bicycle collision prevention. IEEE Control Syst Mag 2021;41(3):28–57. http://dx.doi.org/10.1109/MCS.2021.3062955.

[2] Astrom K, Klein R, Lennartsson A. Bicycle dynamics and control: adapted bicycles for education and research. IEEE Control Syst Mag 2005;25(4):26–47. http://dx.doi.org/10.1109/MCS.2005.1499389.

[3] Maggiore M. On the teaching of nonlinear control: Challenges and ideas. IEEE Control Syst Lett 2023;7:538–44. http://dx.doi.org/10.1109/LCSYS.2022.3200913.

[4] He K, Deng Y, Wang G, Sun X, Sun Y, Chen Z. Learning-based trajectory tracking and balance control for bicycle robots with a pendulum: A Gaussian process approach. IEEE/ASME Trans Mechatronics 2022;27(2):634–44. http://dx.doi.org/10.1109/TMECH.2022.3140885.

[5] Seekhao P, Tungpimolrut K, Parnichkun M. Development and control of a bicycle robot based on steering and pendulum balancing. Mechatronics 2020;69:102386.

[6] Owczarkowski A, Horla D, Zietkiewicz J. Introduction of feedback linearization to robust LQR and LQI control–analysis of results from an unmanned bicycle robot with reaction wheel. Asian J Control 2019;21(2):1028–40.

[7] Chiu C-H, Wu C-Y. Bicycle robot balance control based on a robust intelligent controller. IEEE Access 2020;8:84837–49. http://dx.doi.org/10.1109/ACCESS.2020.2992792.

[8] Chen L, Yan B, Wang H, Shao K, Kurniawan E, Wang G. Extreme-learning-machine-based robust integral terminal sliding mode control of bicycle robot. Control Eng Pract 2022;121:105064. http://dx.doi.org/10.1016/j.conengprac.2022.105064.

[9] Yadav UK, Meena VP, Singh VP. A novel rank-order-centroid based reduction of self-balanced-bicycle-robot controller using grey-wolf optimizer. J Intell Robot Syst 2022;106(3):62. http://dx.doi.org/10.1007/s10846-022-01758-2.

[10] Hartono R, Bin Kim S, Kalend PTA, Baik NK, Shin KJ. Design of self-balancing model-size electrical motorbike robot using control moment gyroscope. Sens Mater 2023;35(2, 1, SI):347–63. http://dx.doi.org/10.18494/SAM4168.

[11] Zhu X, Deng Y, Zheng X, Zheng Q, Chen Z, Liang B, et al. Online series-parallel reinforcement-learning- based balancing control for reaction wheel bicycle robots on a curved pavement. IEEE Access 2023;11:66756–66. http://dx.doi.org/10.1109/ACCESS.2023.3268524.

[12] Wang P, Yi J, Liu T. Stability and control of a rider–bicycle system: Analysis and experiments. IEEE Trans Autom Sci Eng 2020;17(1):348–60. http://dx.doi.org/10.1109/TASE.2019.2922068.

[13] Zheng X, Zhu X, Chen Z, Sun Y, Liang B, Wang T. Dynamic modeling of an unmanned motorcycle and combined balance control with both steering and double CMGs. Mech Mach Theory 2022;169:104643. http://dx.doi.org/10.1016/j.mechmachtheory.2021.104643.

[14] Han F, Huang X, Wang Z, Yi J, Liu T. Autonomous bikebot control for crossing obstacles with assistive leg impulsive actuation. IEEE/ASME Trans Mechatronics 2022;27(4):1882–90. http://dx.doi.org/10.1109/TMECH.2022.3172909.

[15] Han F, Jelvani A, Yi J, Liu T. Coordinated pose control of mobile manipulation with an unstable bikebot platform. IEEE/ASME Trans Mechatronics 2022;27(6):4550–60. http://dx.doi.org/10.1109/TMECH.2022.3157787.

[16] Huang C-F, Tung Y-C, Lu H-T, Yeh T-J. Balancing control of a bicycle-riding humanoid robot with center of gravity estimation. Adv Robot 2018;32(17):918–29.

[17] Sun Y, Zhao H, Chen Z, Zheng X, Zhao M, Liang B. Fuzzy model-based multi-objective dynamic programming with modified particle swarm optimization approach for the balance control of bicycle robot. IET Control Theory Appl 2022;16(1):7–19.

[18] Cui L, Wang S, Zhang Z, Jiang Z-P. Asymptotic trajectory tracking of autonomous bicycles via backstepping and optimal control. IEEE Control Syst Lett 2022;6:1292–7. http://dx.doi.org/10.1109/LCSYS.2021.3091917.

[19] Zhang Y, Zhao G, Li H. Multibody dynamic modeling and controlling for unmanned bicycle system. ISA Trans 2021;118:174–88. http://dx.doi.org/10.1016/j.isatra.2021.02.014.

[20] Stasinopoulos S, Zhao M, Zhong Y. Simultaneous localization and mapping for autonomous bicycles. Int J Adv Robot Syst 2017;14(3):1729881417707170. http://dx.doi.org/10.1177/1729881417707170.

[21] Persson N, Andersson T, Fattouh A, Ekström MC, Papadopoulos AV. A comparative analysis and design of controllers for autonomous bicycles. In: 2021 European control conference. 2021, p. 1570–6. http://dx.doi.org/10.23919/ECC54610.2021.9655223.

[22] Guo L, Chen Z, Song Y. Semi-empirical dynamics modeling of a bicycle robot based on feature selection and RHONN. Neurocomputing 2022;511:448–61. http://dx.doi.org/10.1016/j.neucom.2022.09.062.

[23] Chen B-S, Hung T-W. Integrating local motion planning and robust decentralized fault-tolerant tracking control for search and rescue task of hybrid UAVs and biped robots team system. IEEE Access 2023;11:45888–909. http://dx.doi.org/10.1109/ACCESS.2023.3273787.

[24] Huang Y, Liao Q, Guo L, Wei S. Simple realization of balanced motions under different speeds for a mechanical regulator-free bicycle robot. Robotica 2015;33(9):1958–72. http://dx.doi.org/10.1017/S026357471400112X.

[25] Kiumarsi B, Vamvoudakis KG, Modares H, Lewis FL. Optimal and autonomous control using reinforcement learning: A survey. IEEE Trans Neural Netw Learn Syst 2018;29(6):2042–62. http://dx.doi.org/10.1109/TNNLS.2017.2773458.

[26] Zhu Y, Zhao D. Comprehensive comparison of online ADP algorithms for continuous-time optimal control. Artif Intell Rev 2018;49(4):531–47.

[27] Liu D, Xue S, Zhao B, Luo B, Wei Q. Adaptive dynamic programming for control: A survey and recent advances. IEEE Trans Syst Man Cybern Syst 2021;51(1):142–60. http://dx.doi.org/10.1109/TSMC.2020.3042876.

[28] Huang M, Gao W, Wang Y, Jiang Z-P. Data-driven shared steering control of semi-autonomous vehicles. IEEE Trans Hum-Mach Syst 2019;49(4):350–61. http://dx.doi.org/10.1109/THMS.2019.2900409.

[29] Lin M, Zhao B, Liu D. Policy gradient adaptive critic designs for model-free optimal tracking control with experience replay. IEEE Trans Syst Man Cybern Syst 2022;52(6):3692–703. http://dx.doi.org/10.1109/TSMC.2021.3071968.

[30] Yang Y, Modares H, Vamvoudakis KG, He W, Xu C-Z, Wunsch DC. Hamiltonian-driven adaptive dynamic programming with approximation errors. IEEE Trans Cybern 2022;52(12):13762–73. http://dx.doi.org/10.1109/TCYB.2021.3108034.

[31] Wen G, Hao W, Feng W, Gao K. Optimized backstepping tracking control using reinforcement learning for quadrotor unmanned aerial vehicle system. IEEE Trans Syst Man Cybern Syst 2022;52(8):5004–15. http://dx.doi.org/10.1109/TSMC.2021.3112688.

[32] Lee J, Sutton RS. Policy iterations for reinforcement learning problems in continuous time and space — Fundamental theory and methods. Automatica 2021;126:109421. http://dx.doi.org/10.1016/j.automatica.2020.109421.

[33] Yang R, Wang D, Qiao J. Policy gradient adaptive critic design with dynamic prioritized experience replay for wastewater treatment process control. IEEE Trans Ind Inf 2022;18(5):3150–8. http://dx.doi.org/10.1109/TII.2021.3106402.

[34] Bian T, Jiang Y, Jiang Z-P. Adaptive dynamic programming and optimal control of nonlinear nonaffine systems. Automatica 2014;50(10):2624–32. http://dx.doi.org/10.1016/j.automatica.2014.08.023.

[35] Bian T, Jiang Z-P. Reinforcement learning and adaptive optimal control for continuous-time nonlinear systems: A value iteration approach. IEEE Trans Neural Netw Learn Syst 2022;33(7):2781–90. http://dx.doi.org/10.1109/TNNLS.2020.3045087.

[36] Kiumarsi B, Kang W, Lewis FL. H infinity control of nonaffine aerial systems using off-policy reinforcement learning. Unmanned Syst 2016;04(01):51–60. http://dx.doi.org/10.1142/S2301385016400069.

[37] Fujimoto S, van Hoof H, Meger D. Addressing function approximation error in actor-critic methods. In: Dy J, Krause A, editors. Proceedings of the 35th international conference on machine learning. Proceedings of machine learning research, vol. 80, PMLR; 2018, p. 1587–96.

[38] Vamvoudakis KG, Lewis FL. Online actor–critic algorithm to solve the continuous-time infinite horizon optimal control problem. Automatica 2010;46(5):878–88.

[39] Modares H, Lewis FL, Naghibi-Sistani M-B. Integral reinforcement learning and experience replay for adaptive optimal control of partially-unknown constrained-input continuous-time systems. Automatica 2014;50(1):193–202. http://dx.doi.org/10.1016/j.automatica.2013.09.043.

[40] Vamvoudakis KG, Kokolakis N-MT. Synchronous reinforcement learning-based control for cognitive autonomy. Found Trends Syst Control 2020;8(1–2):1–175. http://dx.doi.org/10.1561/2600000022.

[41] Guo X, Yan W, Cui R. Reinforcement learning-based nearly optimal control for constrained-input partially unknown systems using differentiator. IEEE Trans Neural Netw Learn Syst 2020;31(11):4713–25. http://dx.doi.org/10.1109/TNNLS.2019.2957287.

[42] Kokolakis N-MT, Vamvoudakis KG. Online learning-based optimal control of nonlinear systems with finite-time convergence guarantees. In: 2022 American control conference. 2022, p. 812–7. http://dx.doi.org/10.23919/ACC53348.2022.9867808.

[43] Zhang Y, Mu C, Feng Y, Zhao Z. Improved off-policy reinforcement learning algorithm for robust control of unmodeled nonlinear system with asymmetric state constraints. Internat J Robust Nonlinear Control 2022;1–26. http://dx.doi.org/10.1002/rnc.6432.

[44] Zheng Q, Tian Y, Deng Y, Zhu X, Chen Z, Liang B. Reinforcement learning-based control of single-track two-wheeled robots in narrow terrain. Actuators 2023;12(3):109. http://dx.doi.org/10.3390/act12030109.

[45] Vamvoudakis KG, Vrabie D, Lewis FL. Online adaptive algorithm for optimal control with integral reinforcement learning. Internat J Robust Nonlinear Control 2013;24(17):2686–710.

[46] Abu-Khalaf M, Lewis FL. Nearly optimal control laws for nonlinear systems with saturating actuators using a neural network HJB approach. Automatica 2005;41.